TypeScript

TypeScript Overview

- TypeScript is a strongly typed superset of Javascript that compiles to plain Javascript. It's designed for large-scale Javascript application development and cotains all the elements of Javascript.

- TypeScript is the ES6 and up (ES2019, ESNext etc.) version of Javascript with additional features, developed and maintained by Microsoft under the Apache 2 license.

- TypeScript cannot run directly in the browser, instead the TypeScript code we write gets compiled into Javascript code, which can run directly in the browser. Any valid .js file can be renamed to .ts and compiled with other TypeScript files.

Why Learn TypeScript

- TypeScript provides us with a number of benefits beyond Javascript.
- TypeScript is simple, fast, and easy to learn.
- TypeScript supports all Javascript libraries.
- TypeScript is a safer approach to JavaScript.
- TypeScript is portable, it can run on any environment Javascript runs on.
- TypeScript supports OOP features like classes, inheritance, interfaces, generics etc.
- TypeScript provides compile time error-checking.
- Strict type checking
- Type annotations
- Type inference

TypeScript Environment Setup

- The Typescript Environment
- Install Node.js
- Install TypeScript
- Setup Microsoft Visual Studio Code for TypeScript

Install Node.js

- Node.js comes bundled with the Node Package Manager (npm), installing Node will install npm as well.
- Windows:
  - Point your browser to the Node.js Downloads page.
  - Ensure that you are selecting from the LTS (Long Term Support) tab.
  - Choose either the 32 bit or 64 bit .msi installer based on whichever version of Windows you're running .
  - Once the download has finished, launch the installer and follow the steps in the installation wizard.

Install TypeScript

- Install TypeScript using Node.js Package Manager (npm).Node.js Package Manager (npm)
- Open your Terminal / Command Prompt / Powershell and type the following command.
- npm install –g typescript

TypeScript Basic Syntax

- What is Syntax
- Comments
- Identifiers (names)
- Keywords
- Scope
- Brace Convention

What is Syntax?

- Syntax is the way we write code. It defines a set of rules for developers, and every programming language defines its own syntax.
- Example:

```
let message : string =“Hello World”;
Console.log(message);
```

Comments

- Comments allow us to document our code and enhance its readability.
- Comments ignored by the compiler, which means the compiler won't try to execute any text or code written inside a comment.
- There are many situations where we should make use of comments.
- When learning programming, or a new language, it's helpful to comment our understanding of the code.
- When working on large projects as part of a group, we should provide comments so that other developers immediately understand what's going on.
- When using external libraries it may be useful to include short comments so it's not neccesary to visit the library's documentation that often.

Comments

- When we're working with complex code, it may not be immediately clear what the code is supposed to do. Commenting would make it easier if we need to come back to the code at a later time.
- When we're debugging/testing. We may want to temporarily remove pieces of code from execution by commenting them out.

- TypeScript supports both block comments and C++ style single line comments.

Identifiers(names)

- An identifier is the name we use to identify things like variables, functions etc. There are a few rules and conventions when it comes to naming our identifiers, let's go through some of them now.
- A name cannot start with a numerical value, but may contain a number inside of it.
- A name may start with uppercase or lowercase alphabetical letters (A - Z or a - z), underscores (_), and the dollar symbol ($).
- A name may not contain special characters such as @, % etc.
- Most names are case sensitive, which means that a name with lowercase letters is not the same as a name with uppercase letters.
- A name should not contain a TypeScript/Javascript specific keyword. Keywords are words that are reserved for special uses.
- The convention in TypeScript is to use PascalCase or camelCase to name identifiers.

Keywords

- Keywords are reserved words that have a special meaning to TypeScript

Scope

- TypeScript uses open and close curly braces to define local scope. Everything inside the curly braces is of that scope.
- Example:

```
function logger()
{
let msg =“Hello”;
}
```

Brace Convention

- In TypeScript we're allowed to place our open curly braces on the same line as the header, or on its own line.

```
// TypeScript/Javascript Convention
function logger() {
let msg = "Hello World";
}
// C++ style convention
function logger()
{
  let msg = "Hello World";
}
```

Compiler Flag

- Compiler flags are keywords that we add to our compile command to change the behavior of the compiler when it compiles our code.
- The following table lists some common flags for the TSC compiler.

Compiler Flag

Flag	Description
--help	Shows the help manual
--watch	Watch for changes in the code and recompile them on-the-fly
--init	Initialize a TypeScript project and create a tsconfig.json file
--target	Set the target ECMA version (default is ES3)
--noImplicitAny	Disallow the compiler from inferring the 'any' type
--outFile	Compile multiple files into a single file
--removeComments	Removes all comments from the output file
--build	Build this project and all its dependencies. *This flag is not compatible with other flags*

TypeScript Variables and Constants

- A variable is a temporary data container that stores a value in memory while the application is running.

We can create the variable in one of two ways, with a type, or without.

Syntax:

```
// with type
let name : type;
// without type
let name;
```

Example:

```
// with type
let msg1 : string;
// without type
let msg2;
```

How to assign value to a variable

Now that we have variables, we can store values in them. This is known as assignment.

To assign a value to a variable, we write the name of the variable, followed by the = operator and lastly, the value we want to store.

Syntax: name = value;

Example:

```
    // declaration
    let msg1 : string;
    let msg2;
// assignment
    msg1 = "Hello, World!";
    msg2 = "Hello there";
```

How to initialize a variable with value

We don’t have to declare a variable and assign a value to it afterwards, we can do both in a single step. This is known as initialization.

There are two keywords we can use when we initialize a variable with a value, var and let.

Syntax:

```
// var with type
var name: type = value;
// var without type
var name = value;
// let with type
let name : type = value;
// let without type
let name = value;
```

What is Constant?

- A constant is a variable that cannot have it’s value changed during runtime.

- We use constants when we do not want a value to accidentally change, like the value of Pi.
- A constant cannot be declared without a value because a value cannot be assigned later on, and there would be no point to an empty constant.

- To initialize a constant with a value, we use the const keyword.
- Constants are the same as variables, except their values are
- immutable, they cannot change during runtime.

What is Constant?

Syntax:

```
// const with type
const NAME : type = value;
// const without type
const NAME = value;
```

Example:

```
const PI : number = 3.14159;
const RADIUS = 1.6;
// area of circle
let area = PI * RADIUS * RADIUS;
// print
console.log("Area: ", area);
```

Var vs Let : What is the Difference?

- The important difference between var and let is that let can only be used in a local scope.

- DEMO

TypeScript DataType

- Data types are the different types of values that can be stored by TypeScript.
- The type of a value is checked before the data is stored or manipulated, ensuring that the code will behave as we would expect it to.
- Note that although explicit data type declaration is optional in TypeScript, we recommend it in most cases.

- TypeScript provides us Built-in types and user defined data types

TypeScript DataType

- Built-in types
- Number
- String
- Boolean
- Void
- Null
- Undefined
- Any

Difference between Null & Undefined

Null	Undefined
Null is the intentional absence of a value (null is explicit)	Undefined is the unintentional absence of a value (undefined is implicit)
Null must be assigned to a variable	The default value of any unassigned variable is undefined.
The typeof null is an object. (and not type null)	Typeof undefined is undefined type
You can empty a variable by setting it to null	You can Undefine a variable by setting it to Undefined
null is equal to undefined when compared with == (equality check)	
null is not equal to undefined when compared with === (strict equality check)	
When we convert null to a number it becomes zero	when we convert undefined to number it becomes NaN

TypeScript DataType

- User defined
- Array
- Tuple
- Enumeration (enum)
- Class
- Interface

Operators

- Operators are one or more symbols in TypeScript that is special to the compiler and allows us to perform various operations within our application.
- Types in TypeScript fall under the following categories:
- Arithmetic
- Assignment
- Comparison (Relational)
- Logical (Conditional)
- Other
  - Negation
  - Concatenation
  - Type Of
  - Ternary

Conditional Control Statement

- We can control the flow of our application by evaluating certain conditions. Based on the result of the evaluation, we can then execute certain sections of code.
- We can use conditional control in TypeScript through the following statements, in combination with comparison and logical operators.
  - if
  - else if
  - else
  - switch

Iteration Control Statement

- We can control the flow of our application even further by looping through sections of code when a condition proves true.
- TypeScript provides us with three different kinds of loops:

  - while - This loop iterates through a section of code while a condition is true.
  - do while - This loop is the same as the while loop but with a single forced loop at the start.
  - for - This loop iterates through a section of code a set number of times.

Functions

- Functions allow us to group sections of our code into a smaller, reusable unit.

How to create (define) a named function

- A function consists of the following parts.
  The function keyword.
  A name.
  A parameter list (optional).
  A body with one or more statements.
  A returned value (optional).
- Syntax:

```
function name(parameters : parameter_types) : return_type {
// one or more logic statements

  return value;
}
```

Arrow Functions(Function Expression)

- A function expression, more commonly known as an arrow function, is a shorthand method of writing a function.

- These are similar to lambdas/anonymous functions in other languages like C#.
- Syntax:

```
// definition
var name = (parameters) : return_type => {
  // function body
return value;
}
```

- // call

```
name(parameters);
```

Arrays

- An array is a data container that can store multiple separate values of the same type in a single container.
- How to initialize an array literal
- The easiest, and most straightforward, way to create an array is by initializing an array literal.
- To create an array literal, we start with a unique name, followed by the assignment operator and open and close square brackets.
- Inside the square brackets we add one or more values, separating them with a comma.
- Syntax:
- var name : type[] = [value1, value2, ...];

OOPs Concept

- Abstraction
- Encapsulation
- Inheritance
- Ploymorphism

Classes and Objects

- Class and Object are the basic concepts of Object-Oriented Programming which revolve around the real-life entities. A class is a user-defined blueprint or prototype from which objects are created. Basically, a class combines the fields and methods(member function which defines actions) into a single unit.
- Syntax for class

```
class ClassName {

  // properties
  // methods

}
```

Classes and Objects

- Synatx for Object:
  - var object_name = new ClassName();

How to declare class expressions

- Similar to function expressions, we can declare class expressions. We also use variables to fetch the class, so the class itself is allowed to be either named or unnamed.
- Syntax for unnamed class

```
var expression_name = class {

  // class body
}
```

Syntax for unnamed class

```
var expression_name = class ClassName {
// class body
}
```

How to declare class expressions

- When we instantiate the class expression directly with the new keyword, we must remember to add open and close parentheses behind the expression_name.
- The parentheses are used to construct the object, if there is a constructor present in the declaration.
- When a class expression is named, we cannot use the class name to access its members. We must use the variable name.

How to declare class expressions

Example:

```
var Person = class Employee {
// attributes
  name = "Unknown";
// methods
  walk() {
    document.write("Walking...");
  }
}
// correct
new Person().walk();
// incorrect, will raise an error
new Employee().walk();
```

How to define a class constructor method

- We can create a method that will allow us to populate properties at the same time as we create an object.
- A constructor method uses two special keywords.
    1. constructor
    2. this
- Example

```
class Person {
  firstName;
  lastName;
constructor(fn:string, ln:string) {
    this.firstName = fn;
    this.lastName = ln;
  }
}
var p1 = new Person("John", "Doe");
console.log(p1);
```

Points to Remember (Classes & Object)

- Classes in TypeScript should be compiled with the –target es6 or greater flag.
- A class in an entity that allows us to group both data and behavior into a single unit.
- An object is an instance of the class blueprint. If a class is a cookie cutter, an object would be the cookie.
- A class has to be declared at any stage above its invocation (use).
- We instantiate a class by creating a new instance object.
- We access the members of a class through dot notation, where the object name is followed by a dot and then the member we want to access.
- We can declare class expressions with or without a name.

Points to Remember (Classes & Object)

- We can instantiate a class expression directly with the new keyword, or by creating an object.
- When a class expression is named, we cannot use the name to invoke the class expression. Instead we must use the expression name.
- To create a class constructor we define a method called ‘constructor’ in the class with parameters for the properties we want to initialize with data.
- The properties that we want to initialize must be declared before we try to use them in the constructor.
- The this keyword refers to the calling object.

standalone objects

- Because Javascript is template based, we can create objects in TypeScript directly, instead of needing classes first.

- There are 2 ways to create objects.

- Object Literals
- Instantiating an object with a constructor method and new

How to create an object literal

- We create an object literal by assigning a code block to a variable.
- Inside the code block, we add the property, followed by a colon operator and the value we want to assign to it. If we define multiple properties, they're separated with a comma.
- Syntax:

```
var person = {
  firstName: "John",
  lastName: "Doe",
  age: 32
}
```

- console.log("Name: " + person.firstName + " " + person.lastName);
- console.log("Age: " + person.age);

How to instantiate a new object with a constructor method

- A constructor is a function that initializes the object with values we specify.

- An object constructor is a little bit different than a class constructor

- First, we need to create a function with all the properties we want to initialize when the object is instantiated, as function parameters.
- Synatx:

```
function constructor_name(property1, property2, ...) {}
```

How to define a method within a standalone object

- We can define functions inside a standalone object. But, before we define the function, we must first create a variable with the same name as the function and refer to it with the this keyword.
- Syntax:

```
function constructor_name() {


  this.method_name = function_name;


  function_name() {
        // body
  }
}
```

How to pass an object to a function as parameter

- We can pass an object to a function as an argument, but we should also add the properties we want available to that object.
- Syntax:

```
var object_name {

        property_name: property_value;
}
function func_name(obj_param: { property_name:property_type } ) :
return_type {

        // use obj_param.property
}
```

Static Methods

- A static method is a method that belongs to the class itself, not the instance of a class.
- That means we don’t need an object to call a static class method. We call them directly from the class itself.
- In a real world situation, static methods would typically be used for helper, or utility methods
- we specify one of the following three access modifiers before the static keyword.

1. Public(By default)
2. private
3. protected

Static Methods

- We are not limited to using only static methods. TypeScript allows us to declare our properties as static as well.

- Static properties are also defined with the static keyword in front of the property name, and are accessed directly on the class with dot notation.
- Example:

```
class Person {

  static age = 28;
}

console.log(Person.age);
```

Inheritance

- Inheritance makes it possible to reuse code from other classes.
- Developers can wrap common sections of code in one class, then inherit this functionality and reuse that code in a sub, or child class.
- The main class is known as the base, parent or super class. The class that inherits code from the parent is known as the sub, or child class.
- Inheritance is referred to as an is-a type relationship between classes. For example, a cat is an animal, or a car is a vehicle.
- Inheritance also provides us with polymorphic behavior, which can be quite powerful.

Inheritance

- Inheritance allows one class to inherit functionality from another without rewriting the same code.
- We use the extends keyword to indicate that one class inherits from another.
- The child class will automatically receive all the functionality from the parent class unless it contains its own constructor.
- We inherit constructor functionality with the super() constructor.
- If the parent class constructor has any parameters, the super() constructor will need the same parameters.

Inheritance

- An interface is like a class whose members aren't implemented.
- Interfaces allow us to have easy has-a relationships between classes. Instead of defining a class for each relationship, we define interfaces.
- Inheritance makes our classes tightly coupled and dependent on each other. We want them to be as loosely coupled as possible.
- Interfaces are define the same way as classes, except we use the interface keyword instead of the class keyword.
- Interface members aren't implemented. We don't initialize properties with values, or add code blocks to methods, although they can contain parameters.
- Interfaces allow for a loosely coupled has-a relationship between classes.

Inheritance

- Syntax:

```
interface IInterfaceName {
  property:type;
  method(parameter:type);
}
```

- A popular convention is to prefix the interface name with the capital letter I, to know at a glance that it's an interface.
- Example

```
interface IFlyable {
// properties
  maxSpeed:number;
  // methods
  fly(speed);
}
```

Exception Handling

- Exception:
  - Exceptions are used to indicate that a program or module is unable to continue processing. By their very nature, they should only be raised in truly exceptional circumstances.
  - Often, exceptions are used to indicate that the state of the program is invalid and is not safe to continue. Although it can be tempting to start issuing exceptions every time a routine is passed a disagreeable value as an argument, it can often be more graceful to handle input that you can anticipate without raising an exception.
  - When your program encounters an exception, it will be shown in the JavaScript console unless it is handled in code. The console allows programmers to write messages, and it will automatically log any exceptions that occur while running the program

Exception Handling

- Throwing Exception:
  - To raise an exception in your TypeScript program you use the throw keyword. Although you can follow this keyword with any object, it is best to provide either a string containing an error message, or an instance of the Error object wrapping the error message.
- You can create a custom exception using a class that implements the Error interface
- The Error interface ensures that your class has a name and message property.
- By adding the toString method to the custom exception class you can ensure that an appropriate message is shown when the exception is thrown and logged.

Asynchronous

- In general, asynchronous -- pronounced ay-SIHN-kro-nuhs, from Greek asyn-, meaning "not with," and chronos, meaning "time" -- is an adjective describing objects or events that are not coordinated in time.

- What does asynchronous mean?

- More specifically, asynchronous describes the relationship between two or more events/objects that do interact within the same system but do not occur at predetermined intervals and do not necessarily rely on each other's existence to function. They are not coordinated with each other, meaning they could occur simultaneously or not because they have their own separate agenda.

Asynchronous in computer programming

- In computer programming, asynchronous operation means that a process operates independently of other processes,
- whereas synchronous operation means that the process runs only as a result of some other process being completed or handed off.
- A typical activity that might use a synchronous protocol would be a transmission of files from one point to another. As each transmission is received, a response is returned indicating success or the need to resend. Each successive transmission of data requires a response to the previous transmission before a new one can be initiated.
- asynchronous programming provides opportunities for a program to continue running other code while waiting for a long-running task to complete. The time-consuming task is executed in the background while the rest of the code continues to execute.

Asynchronous in computer programming

- The feature that enables asynchronous programming in these languages is referred to as a callback function. In JavaScript, all operations nested in the function are sent off to a web application or database to gather the necessary information while the rest of the program continues to run. When the information is gathered, it is sent back through the program and applied to the functions in the program that rely on it, hence callback.

async/await

- async: The async keyword is used to define a function as asynchronous. An asynchronous function returns a Promise implicitly, allowing you to use await within it. When a function is marked as async, it means that it will always return a Promise.

```
async function fetchData() {
    // Asynchronous operations here
}
```

async/await

- await: The await keyword can only be used inside an async function. It is used to pause the execution of the async function until a Promise is resolved, and then returns the resolved value. If the Promise is rejected, it throws an error that you can handle with a try...catch block.

async/await

- Promise:a Promise is a built-in JavaScript object representing the eventual completion or failure of an asynchronous operation, and its resulting value.
- When you make an asynchronous operation, such as an HTTP request or a timer, it doesn't block the execution of the rest of your code. Instead, it continues executing, and when the asynchronous operation finishes, it triggers a callback function. Promises provide a way to handle these asynchronous operations more cleanly and with better error handling.

async/await

- States: A Promise can be in one of three states:
- Pending: Initial state, neither fulfilled nor rejected.
- Fulfilled: The operation completed successfully.
- Rejected: The operation failed.

Thank You